

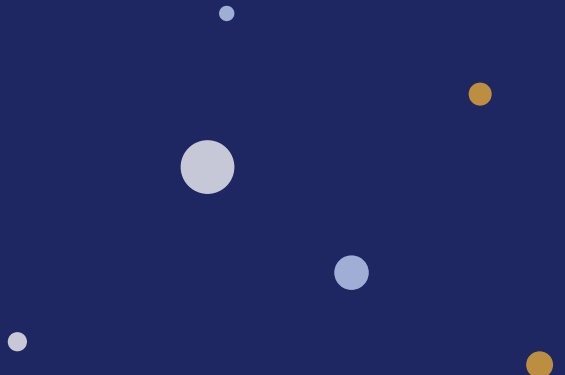


COMPLETE STUDY GUIDE

Pandas for Data Analysis

SQL-to-Pandas Edition — Data structures, manipulation, cleaning, analysis, visualization, and I/O, with a full cheatsheet.

Companion deck: "Pandas for SQL Professionals — 10-Day Study Plan"



READ THIS FIRST

How to Use This Guide

- Organized into six topic sections mirroring what you set out to learn, followed by a SQL-to-Pandas translation reference and a quick-reference cheatsheet.
- Each section opens with core concepts, moves through worked examples, and ends with practice exercises for your own domain tables.
- Use it as a living reference during the 10-Day Study Plan, and afterward as a desk reference — the cheatsheet at the end is built for that.
- **Code examples assume: `import pandas as pd`, `import numpy as np`**

The SQL-to-Pandas Mental Model

SQL Concept	Pandas Equivalent	Notes
Table	DataFrame	2-D labeled structure
Column (typed)	Series	1-D labeled array, one dtype
Row / primary key	Index	Explicit; can be reset or composed
Schema / column types	.dtypes	object, int64, float64, datetime64, category
NULL	NaN / NaT / None	NaN propagates differently in comparisons
WHERE	Boolean mask, .query()	
GROUP BY / HAVING	.groupby().agg(), then filter	
JOIN	.merge()	how='inner'/'left'/'right'/'outer'
UNION / UNION ALL	pd.concat()	+.drop_duplicates() for UNION
ORDER BY	.sort_values()	
Window functions	.rolling()/.rank()/.transform()	
PIVOT / UNPIVOT	.pivot_table() / .melt()	

GET READY

Environment Setup

INSTALL & VERIFY

```
pip install pandas numpy matplotlib seaborn \
    jupyterlab "snowflake-connector-python[pandas]"

import pandas as pd
print(pd.__version__) # this guide targets pandas 2.x
```

- Recommended environment: JupyterLab or VS Code + Jupyter extension — both give inline output like a SQL client's results grid.
- Snowflake access: have your account identifier, warehouse, database, schema, and SSO or key-pair auth ready before the I/O section.

Tip: pin your pandas version in any shared notebook — behavior has shifted meaningfully across 1.x → 2.x releases.

01

Data Structures — Series & DataFrame

What a Series and DataFrame actually are, how the Index works, dtypes, inspection methods, and the three ways to select data.

1. DATA STRUCTURES

The Series

CREATE & USE A SERIES

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
```

```
s['b']          # 20 – access by label  
s.iloc[1]       # 20 – access by position  
s * 2           # vectorized: [20, 40, 60]  
s > 15          # vectorized boolean mask
```

- A Series is a one-dimensional labeled array: values + an index, all one dtype.
- Vectorized operations apply element-wise, no loops.
- Think of it as one column of a SQL result set that carries its own row labels wherever it goes — even after filtering.

1. DATA STRUCTURES

The DataFrame

CREATE A DATAFRAME

```
pd.DataFrame({'id':[1,2,3], 'name':['A','B','C']})

pd.read_csv('file.csv')
pd.read_sql(sql, con)          # straight from SQL /
Snowflake
```

- A 2-D table: a dict of Series sharing one Index, each column with its own dtype — like a typed SQL result set.
- Every DataFrame exposes: `.index` (row labels), `.columns`, `.dtypes`, `.values` / `.to_numpy()`.
- Create from a dict, list of records, CSV, or directly from SQL.

1. DATA STRUCTURES

The Index, Deep Dive

- The Index is what makes pandas different from a raw SQL result set: a persistent row-label axis, not just a rowid.
- Default is a RangeIndex (0, 1, 2, ...), auto-created if you don't set one.
- Set a meaningful index — `df.set_index('customer_id')` — like designating a primary key for lookups.
- MultiIndex for composite keys — `df.set_index(['region','year'])` — mirrors a composite primary/foreign key. Reset with `df.reset_index()`.
- **Why it matters: joins, alignment, and `.loc` all key off the index — misunderstanding it is the #1 source of "why did my row count change" bugs for SQL veterans.**

1. DATA STRUCTURES

Inspecting Data

Task	Code	SQL analogy
Row/column count	<code>df.shape</code>	<code>SELECT COUNT(*) + column count</code>
Column types	<code>df.dtypes</code>	<code>DESCRIBE TABLE</code>
Full profile	<code>df.info()</code>	<code>dtypes + non-null counts + memory</code>
Summary stats	<code>df.describe()</code>	<code>MIN/MAX/AVG/quartiles in one call</code>
First/last N rows	<code>df.head(n) / df.tail(n)</code>	<code>SELECT * FROM t LIMIT n</code>
Random sample	<code>df.sample(n)</code>	<code>ORDER BY RANDOM() LIMIT n</code>

1. DATA STRUCTURES

Selecting Data: Columns

- Single column (returns a Series): `df['col']` or `df.col`
- Avoid the dot form for columns with spaces or that shadow a method name (e.g. a column literally called 'count').
- Multiple columns (returns a DataFrame): `df[['col1','col2']]` — note the double brackets.
- Equivalent to `SELECT col1, col2 FROM t`.

1. DATA STRUCTURES

Selecting Data: .loc vs .iloc

LABEL-BASED VS POSITION-BASED

```
df.loc[df['status'] == 'active', ['id', 'name']]  
# filter rows AND pick columns in one call
```

```
df.iloc[0:5, 0:3]  
# first 5 rows, first 3 columns, by position
```

- `.loc[row_labels, col_labels]` — label-based, inclusive of both endpoints in slices.
- `.iloc[row_positions, col_positions]` — integer-position-based, exclusive end (like Python slicing).
- Rule of thumb: thinking in names → `.loc`. Thinking in positions → `.iloc`.

Section 1 Exercises

- Load a CSV or table you know well. Run `.info()` and `.describe(include='all')` and note what would otherwise take several separate SQL queries.
- Set a meaningful column as the index, then retrieve a specific row by that key with `.loc` — compare to `SELECT * FROM t WHERE key = :val`.
- Build a small DataFrame with a MultiIndex on two columns and retrieve a slice with `.loc[(val1, val2)]`.

02

Data Manipulation — Filter, Sort, Group, Merge

The section with the most direct SQL overlap: WHERE, ORDER BY, GROUP BY/HAVING, and JOIN — plus reshaping and window-function analogs.

2. DATA MANIPULATION

Filtering Rows (WHERE)

BOOLEAN MASKS & .QUERY()

```
df[df['amount'] > 1000]
```

```
df[(df['amount'] > 1000) & (df['region'] == 'East')]
```

```
df[df['status'].isin(['open', 'pending'])] # IN
```

```
df[df['amount'].between(100, 500)] #
```

BETWEEN

```
df.query("amount > 1000 and region == 'East'") #
```

readable form

```
df[df['name'].str.contains('smith', case=False,  
na=False)] # LIKE '%..%'
```

- Use & | ~, never and/or/not, and parenthesize every condition.
- .isin() = IN, .between() = BETWEEN, .str.contains() = LIKE.

2. DATA MANIPULATION

Sorting (ORDER BY)

SORT_VALUES / SORT_INDEX

```
df.sort_values('amount', ascending=False)
```

```
df.sort_values(['region', 'amount'], ascending=[True,  
False])
```

```
df.sort_index()  
df.sort_values('amount', na_position='first')
```

- Independent ascending/descending direction per key.
- `na_position` controls where NaNs land — like NULLS FIRST/LAST.

2. DATA MANIPULATION

Derived / Computed Columns

COMPUTED SELECT & CASE WHEN

```
df['margin'] = df['revenue'] - df['cost']          #  
vectorized, preferred  
  
df.assign(margin=lambda d: d['revenue'] - d['cost'])  
  
df['flag'] = np.where(df['amount'] > 1000, 'high',  
                     'low')          # binary  
df['tier'] = np.select([c1, c2], ['A', 'B'],  
                      default='C')    # multi-branch  
df['region_name'] =  
df['region_code'].map(region_lookup_dict) # lookup  
join
```

- `.apply()` handles row-wise custom logic when vectorization isn't possible — but it's the slowest option; reach for it last.

2. DATA MANIPULATION

GroupBy Fundamentals (GROUP BY)

SPLIT-APPLY-COMBINE

```
df.groupby('region')['amount'].sum()
```

```
df.groupby(['region', 'product'])  
['amount'].sum().reset_index()
```

```
df.groupby('region').size()    # like COUNT(*) –  
includes NaNs
```

```
df.groupby('region')['amount'].count() # like  
COUNT(col) – non-null only
```

- Split-apply-combine is exactly what GROUP BY does under the hood.
- `.reset_index()` after groupby turns the grouped result back into a flat DataFrame.

2. DATA MANIPULATION

GroupBy: Aggregation Patterns

AGG(), NAMED AGGREGATION, HAVING

```
df.groupby('region').agg({'amount': ['sum', 'mean'],  
                          'qty': 'sum'})
```

```
df.groupby('region').agg(  
    total_amount=('amount', 'sum'),  
    avg_amount=('amount', 'mean'))          # named  
aggregation
```

```
grouped = df.groupby('region')  
['amount'].sum().reset_index()  
grouped[grouped['amount'] > 100000]        # HAVING  
SUM(amount) > 100000
```

- Or filter groups before combining: `.groupby().filter(lambda g: len(g) > 10)` — like `HAVING COUNT(*) > 10`.

2. DATA MANIPULATION

GroupBy: transform() and apply()

ROW-ALIGNED GROUP RESULTS

```
df['region_total'] = df.groupby('region')  
['amount'].transform('sum')  
# every row gets its group's total – like a correlated  
subquery
```

```
df.groupby('region').apply(custom_func)  
# most flexible, any return shape – but the slowest  
option
```

- .transform() returns a result the same shape as the input — perfect for adding a group total beside every row.
- .apply() on a groupby object runs arbitrary logic per group; use it when agg/transform can't express the logic.

Merging & Joining (JOIN)

MERGE() WITH SAFETY NETS

```
pd.merge(orders, customers, on='customer_id',
        how='left')

pd.merge(orders, customers,
        left_on='cust_id', right_on='customer_id',
        how='inner', validate='many_to_one',
        indicator=True)
```

- `how=`: 'inner' / 'left' / 'right' / 'outer' map directly to SQL join types.
- `validate=` raises an error if join cardinality isn't what you expect — catches silent row multiplication before it corrupts a report.
- `indicator=True` adds a `_merge` column ('both'/'left_only'/'right_only') — invaluable for auditing outer joins.

2. DATA MANIPULATION

Concatenating (UNION / UNION ALL)

STACKING ROWS AND COLUMNS

```
pd.concat([df1, df2])                # UNION ALL
– keeps duplicates
pd.concat([df1, df2]).drop_duplicates() # UNION –
removes exact dupes
pd.concat([df1, df2], ignore_index=True) # fresh
RangeIndex

pd.concat([df1, df2], axis=1)         # side-by-
side (columns)
```

- `ignore_index=True` avoids carrying over each source's original index.

2. DATA MANIPULATION

Reshaping: Pivot & Pivot Tables (PIVOT)

LONG TO WIDE, WITH AGGREGATION BUILT IN

```
pd.pivot_table(df, index='region', columns='year',  
               values='amount', aggfunc='sum',  
               fill_value=0)  
  
df.pivot(index='r', columns='c', values='v')    # no  
aggregation needed  
pd.crosstab(df['region'], df['status'])          # quick  
frequency table
```

- This is exactly SQL's PIVOT — but the aggregate is built in, unlike hand-written CASE WHEN columns.

2. DATA MANIPULATION

Reshaping: Melt & Stack/Unstack (UNPIVOT)

WIDE TO LONG

```
df.melt(id_vars=['region'], value_vars=['2024',  
    '2025'],  
        var_name='year', value_name='amount')  
  
df.stack()      # columns -> innermost row index level  
df.unstack()    # the reverse
```

- melt() = SQL's UNPIVOT — use it for tidy, long-format data ready for grouping or plotting.
- Use pivot_table when you want a wide summary for reporting instead.

2. DATA MANIPULATION

Window-Function Equivalents

SQL window function	Pandas equivalent
SUM(x) OVER (PARTITION BY g)	<code>df.groupby('g')['x'].transform('sum')</code>
ROW_NUMBER() OVER (PARTITION BY g ORDER BY x)	<code>df.groupby('g')['x'].rank(method='first')</code>
RANK() OVER (...)	<code>df.groupby('g')['x'].rank(method='min')</code>
LAG(x) OVER (ORDER BY d)	<code>df.sort_values('d')['x'].shift(1)</code>
LEAD(x) OVER (ORDER BY d)	<code>df.sort_values('d')['x'].shift(-1)</code>
Running total	<code>df.sort_values('d')['x'].cumsum()</code>
Moving average (N periods)	<code>df['x'].rolling(window=N).mean()</code>

.rolling() = fixed-size moving window; .expanding() = a window that grows from the start, like a running/cumulative frame.

Section 2 Exercises

- Reproduce a GROUP BY ... HAVING ... report with `.groupby().agg()` plus a post-filter.
- Reproduce a 2-3 table JOIN with `.merge()`, using `validate=` and `indicator=True` to confirm the row count matches your SQL result exactly.
- Build a wide PIVOT-style report with `pivot_table`, then `melt()` it back to long form and confirm it matches your original rows.
- Reproduce one OVER (PARTITION BY ... ORDER BY ...) query using `groupby` + `transform/rank/shift`.

03

Data Cleaning

Missing data, duplicates, outliers, dtype conversions, and string cleaning — the pandas equivalent of ETL scrubbing logic.

3. DATA CLEANING

Missing Data: Detecting and Dropping

ISNA / DROPNA

```
df.isna().sum()           # missing-value count per
column

df.dropna()               # drop rows with any NaN
df.dropna(subset=['col']) # only consider specific
columns
df.dropna(how='all')       # only drop fully-empty
rows
```

- pandas uses NaN (float) / NaT (datetime) for missing values.
- Comparisons behave like SQL NULL (NaN == NaN is False), but arithmetic silently propagates NaN rather than requiring COALESCE.

3. DATA CLEANING

Missing Data: Filling and Interpolating

FILLNA / INTERPOLATE

```
df['col'].fillna(0)                # COALESCE(col,  
0)  
df['col'].fillna(df['col'].mean())  # statistical  
fill  
df['col'].fillna(method='ffill')    # carry last  
known value forward  
df['col'].interpolate()            # linear  
interpolation
```

- Always document why you chose a fill strategy — the same discipline as a COALESCE / default-value decision in an ETL job.

3. DATA CLEANING

Duplicates

DUPLICATED / DROP_DUPLICATES

```
df.duplicated()                # flags  
duplicate rows  
df.duplicated(subset=['id'])   # check a  
would-be unique key  
df.drop_duplicates(subset=['id'], keep='last')
```

- Good habit: run `df.duplicated(subset=key_cols).sum()` right after any merge to catch unexpected fan-out immediately.

3. DATA CLEANING

Outliers

IQR, Z-SCORE, CAPPING

```
Q1, Q3 = df['x'].quantile([0.25, 0.75])
IQR = Q3 - Q1
mask = (df['x'] < Q1 - 1.5*IQR) | (df['x'] > Q3 +
1.5*IQR)

z = (df['x'] - df['x'].mean()) / df['x'].std() # flag
abs(z) > 3
df['x'].clip(lower, upper) # cap
/ winsorize
```

- Decide deliberately whether to drop, cap, or keep-and-flag outliers — and document the choice.

3. DATA CLEANING

Data Type Conversions

ASTYPE / TO_DATETIME / CATEGORY

```
df['col'].astype('int64')
pd.to_datetime(df['date_str'], format='%Y-%m-%d')
pd.to_numeric(df['col'], errors='coerce')    #
unparseable -> NaN, no crash
df['status'] = df['status'].astype('category')
```

- category dtype for low-cardinality text (status, region) — saves memory and speeds up groupby, like a small dimension-table key.

3. DATA CLEANING

String Cleaning

THE .STR ACCESSOR

```
df['name'].str.strip().str.lower()

df['name'].str.replace(' ', ' ', regex=False)
df['col'].str.extract(r'(\d{5})')          # pull a
ZIP code
df['col'].str.split(',', expand=True)      # like
chained SPLIT_PART
```

- The .str accessor vectorizes string operations across an entire column at once.

Section 3 Exercises

- Take a genuinely messy extract (blanks/nulls, inconsistent casing, duplicate rows) and produce a clean DataFrame, documenting each decision inline.
- Apply the IQR method to one numeric column in a domain dataset and report how many rows would be flagged as outliers.
- Extract a structured sub-value (ID, code, or date) out of a free-text column with `.str.extract()`.

04

Data Analysis — Stats, Correlation, Regression

Turning a clean, joined DataFrame into insight: summary statistics, correlation, time-aware analysis, and a first step into regression.

4. DATA ANALYSIS

Descriptive Statistics

DESCRIBE / VALUE_COUNTS

```
df.describe() #  
count/mean/std/min/quartiles/max  
df['x'].mean(); df['x'].median(); df['x'].std()  
  
df['category_col'].value_counts() #  
frequency table  
df['category_col'].value_counts(normalize=True) # as  
proportions
```

- All summary stats skip NaN by default (skipna=True).
- value_counts() replaces a GROUP BY + COUNT(*) + ORDER BY in a single call.

4. DATA ANALYSIS

Correlation & Covariance

CORR / COV

```
df[['x', 'y', 'z']].corr()           #  
Pearson correlation matrix  
df['x'].corr(df['y'], method='spearman')  # rank-  
based, non-linear  
df.cov()                             #  
covariance matrix
```

- Visualize a correlation matrix as a heatmap (Section 5) — far faster to scan than a wall of numbers.

Grouped & Pivoted Analysis

- Combine groupby + agg to answer "how does metric X vary by segment Y" — the bread-and-butter analytical question.
- Layer a pivot_table on top for a reporting-ready wide view (segments as columns, periods as rows, or vice versa).
- `df.groupby('segment')['metric'].describe()` gives a full distribution summary per segment in one call — useful for spotting a skewed segment before drawing conclusions.

4. DATA ANALYSIS

Time Series Basics

RESAMPLE / ROLLING / SHIFT

```
df['date'] = pd.to_datetime(df['date'])  
df = df.set_index('date')
```

```
df['amount'].resample('M').sum()           # monthly  
totals from daily data  
df['amount'].rolling(window=7).mean()      # 7-day moving  
average  
df['amount'] - df['amount'].shift(1)        # period-over-  
period change
```

- `resample()` is a `DATE_TRUNC` + `GROUP BY` in one call, once the index is a proper `DatetimeIndex`.

4. DATA ANALYSIS

A First Look at Regression

STATSMODELS & SCIKIT-LEARN

```
import statsmodels.api as sm
X = sm.add_constant(df[['x1', 'x2']])
model = sm.OLS(df['y'], X).fit()
model.summary()
```

```
from sklearn.linear_model import LinearRegression
LinearRegression().fit(df[['x1', 'x2']], df['y'])
```

- Pandas hands off cleanly to statsmodels/scikit-learn once data is clean and joined — pandas is the prep layer, not the modeling layer.
- Use this to quantify a trend ("cost per claim rising \$X/month") rather than eyeballing a chart.

Section 4 Exercises

- Produce a `.describe()` breakdown by segment for one metric you track regularly, and note which segment has the widest spread.
- Build a correlation matrix for 4–5 numeric columns in a domain dataset and identify the strongest relationship.
- Resample a date-indexed metric to monthly totals and compute month-over-month percent change with `.pct_change()`.

05

Data Visualization

Turning DataFrames into charts: pandas' built-in plotting, matplotlib fundamentals, and seaborn for statistical graphics.

5. DATA VISUALIZATION

Quick Plots Directly from Pandas

DF.PLOT()

```
df['amount'].plot(kind='line')
df.plot(kind='bar', x='region', y='amount')

# kind=: 'line' | 'bar'/'barh' | 'hist' | 'box' |
'scatter' | 'area' | 'pie'
```

- Every DataFrame/Series has a .plot() method built on matplotlib.
- Great for fast, exploratory looks — the visual equivalent of eyeballing a result grid.

5. DATA VISUALIZATION

Matplotlib Fundamentals

THE OBJECT-ORIENTED INTERFACE

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(x, y)
ax.set_title('Monthly Trend')
ax.set_xlabel('Month'); ax.set_ylabel('Amount')

fig.savefig('chart.png', dpi=150, bbox_inches='tight')
```

- Full control: multiple series, secondary y-axis, subplot grids (plt.subplots(nrows=2, ncols=2)) for a small dashboard.

5. DATA VISUALIZATION

Seaborn for Statistical Graphics

WORKS DIRECTLY ON DATAFRAME COLUMNS

```
sns.barplot(data=df, x='region', y='amount',  
hue='year')  
sns.histplot(data=df, x='amount', kde=True)  
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')  
sns.boxplot(data=df, x='segment', y='metric')
```

- Built on matplotlib, but the API reads almost like a SELECT — column names as arguments.
- boxplot compares distributions across categories and spots outliers visually.

5. DATA VISUALIZATION

Choosing the Right Chart

Question	Chart
Trend over time	Line chart
Compare categories	Bar chart
Distribution of one variable	Histogram / box plot
Relationship between two numeric variables	Scatter plot
Correlation across many variables	Heatmap
Part-to-whole	Stacked bar (avoid pie for >4 categories)

Section 5 Exercises

- Build a bar chart comparing a metric across your top 5–10 categories, sorted descending before plotting.
- Build a line chart of a metric over time with a 7-period rolling average overlaid.
- Build a seaborn heatmap of the correlation matrix from your Section 4 exercise.

06

Data I/O

Flat files, and — most relevant to your day job — SQL and Snowflake, plus the Snowpark pandas API for running pandas logic at scale.

Flat Files: CSV, Excel, JSON

READ_CSV / READ_EXCEL / READ_JSON

```
pd.read_csv('file.csv', dtype={...},
parse_dates=['date_col'], usecols=[...])
pd.read_excel('file.xlsx', sheet_name='Sheet1')  #
needs openpyxl
pd.read_json('file.json')
pd.json_normalize(nested_data)                  #
flatten nested JSON

df.to_csv('out.csv', index=False)
```

- Control dtypes and memory up front rather than fixing them after load.
- `json_normalize` is pandas' answer to flattening a semi-structured column.

Reading From SQL and Snowflake

READ_SQL / FETCH_PANDAS_ALL

```
pd.read_sql("SELECT * FROM orders WHERE region =  
'East'", con=connection)
```

```
cur.execute(sql)  
df = cur.fetch_pandas_all()    # fastest path from  
Snowflake — uses Arrow
```

- Generic `read_sql` works with any DB-API/SQLAlchemy connection.
- Push filtering/aggregation into the SQL you send to Snowflake for large tables — pull only what you need, just as you would writing a report query today.

Writing Back to Snowflake

WRITE_PANDAS()

```
from snowflake.connector.pandas_tools import  
write_pandas  
write_pandas(conn, df, 'TABLE_NAME')
```

- Bulk-loads a DataFrame into a Snowflake table efficiently.
- Pattern: pull -> clean/join/analyze in pandas -> write results to a "gold" table for others to query with plain SQL.
- Watch for: chunking large writes, and matching target-table identifier casing/quoting.

Pandas on Snowflake: the Snowpark pandas API

- Snowflake also offers a Snowpark pandas API (built on Modin): `import modin.pandas as pd` — familiar pandas syntax, execution pushed down and run inside Snowflake.
- Data never leaves the warehouse — useful once datasets outgrow what's comfortable in local memory.
- Same mental model as everything in this guide; the API surface is designed to match standard pandas closely.
- **Reach for it when your Section 2 `groupby`/`merge`/`pivot` logic needs to run over full production-scale tables rather than a local extract.**

Performance Tips

- Prefer vectorized operations and built-in methods over `.apply()`/Python loops — often 10–100x faster.
- Set dtypes explicitly on load (`dtype=` in `read_csv/read_sql`) rather than inferring then re-casting.
- Use category dtype for repeated low-cardinality strings.
- For very large sources: filter/aggregate in SQL first, read in `chunksizes=` batches, or use the Snowpark pandas API.

Section 6 Exercises

- Load one CSV and one table via `read_sql`/Snowflake connector; confirm inferred dtypes match the source column types.
- Write a small DataFrame to a scratch Snowflake table with `write_pandas`, read it back, and confirm row-for-row equality.
- If Snowpark pandas is available, rerun one Section 2 groupby/merge exercise using `modin.pandas` against a full table instead of a local extract.

07

SQL-to-Pandas Translation & Pitfalls

A side-by-side reference for translating full queries, plus the handful of habits that most often trip up SQL veterans.

Translating SELECT / WHERE / ORDER BY

SQL

```
SELECT customer_id, amount, region
FROM orders
WHERE amount > 500
      AND region = 'East'
ORDER BY amount DESC;
```

PANDAS

```
(orders
 .loc[(orders['amount'] > 500) &
       (orders['region'] == 'East'),
       ['customer_id', 'amount', 'region']]
 .sort_values('amount', ascending=False))
```

Translating GROUP BY / HAVING

SQL

```
SELECT region, SUM(amount) AS total  
FROM orders  
GROUP BY region  
HAVING SUM(amount) > 100000;
```

PANDAS

```
(orders.groupby('region')['amount']  
 .sum()  
 .reset_index(name='total')  
 .query('total > 100000'))
```

Translating a Multi-Table JOIN

SQL

```
SELECT o.order_id, c.customer_name, o.amount
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
LEFT JOIN regions r
  ON c.region_id = r.region_id;
```

PANDAS

```
(orders
  .merge(customers, on='customer_id',
         how='inner', validate='m:1')
  .merge(regions, on='region_id',
         how='left', validate='m:1')
  [['order_id', 'customer_name', 'amount']])
```

Common Pitfalls for SQL Developers

- Index confusion: a filtered DataFrame keeps its original row labels, not 0..n. Use `.reset_index(drop=True)` for a fresh RangeIndex.
- SettingWithCopyWarning: assigning into a filtered slice may silently fail to update the original. Use `.loc[...]` for read-and-write in one step, or `.copy()` for an independent DataFrame.
- NaN vs NULL semantics: arithmetic on NaN silently yields NaN rather than erroring; `dropna()` is an implicit IS NOT NULL you must apply deliberately.
- **Merge cardinality surprises: an unexpected many-to-many join silently multiplies rows with no error — always use `validate=` on unfamiliar tables.**
- Mutating vs copying: most methods return a new object by default; some accept `inplace=True`. Prefer the non-mutating, chainable style.

Section 7 Exercises

- Pick three SQL queries you run weekly and write out their full pandas translations side-by-side.
- Deliberately trigger a `SettingWithCopyWarning`, then fix it using `.loc` — understanding the message is worth five minutes.
- Force a many-to-many merge on sample data with and without `validate=` to see the row-multiplication effect firsthand.

08

Handy References & Links

Every resource cited in this guide and the 10-Day Study Plan, organized for quick lookup.

8. REFERENCES

Official Docs & Cheat Sheets

pandas official documentation pandas.pydata.org/docs

10 Minutes to pandas (official quickstart) pandas.pydata.org/docs/user_guide/10min.html

Official Comparison with SQL pandas.pydata.org/docs/getting_started/comparison/comparison_with_sql.html

Official pandas Cheat Sheet (PDF) pandas.pydata.org/Pandas_Cheat_Sheet.pdf

Official pandas Cookbook pandas.pydata.org/docs/getting_started/intro_tutorials

8. REFERENCES

Free Books & Video Courses

"Python for Data Analysis, 3E" by Wes McKinney — free, open-access, full text wesmckinney.com/book

Keith Galli — Complete Pandas Tutorial (code + video) github.com/KeithGalli/complete-pandas-tutorial · youtu.be/2uvysYbKdjM

freeCodeCamp — Data Analysis with Python (full course) freecodecamp.org/news/how-to-analyze-data-with-python-pandas

Corey Schafer — Pandas Tutorials playlist youtube.com/playlist?list=PL-osiE80TeTsWmV9i9c58mdDCSskIFdDS

Data School (Kevin Markham) — Easier data analysis with pandas dataschool.io/easier-data-analysis-with-pandas

techwithtim — Pandas Tutorial repo github.com/techwithtim/PanadasTutorial

8. REFERENCES

Snowflake & SQL Integration

Snowflake official: Using pandas DataFrames with the Python Connector docs.snowflake.com/en/developer-guide/python-connector/python-connector-pandas

Snowflake official: pandas on Snowflake (Snowpark pandas API) docs.snowflake.com/en/developer-guide/snowpark/python/pandas-on-snowflake

pandas official I/O guide (all formats incl. SQL) pandas.pydata.org/docs/user_guide/io.html



09

Cheatsheet

A six-page quick-reference — keep this open next to your notebook.



Inspecting & Selecting

Task	Code
Shape	<code>df.shape</code>
Dtypes	<code>df.dtypes</code>
Full summary	<code>df.info()</code>
Stats summary	<code>df.describe()</code>
First / last N rows	<code>df.head(n)</code> / <code>df.tail(n)</code>
Select column(s)	<code>df['a']</code> / <code>df[['a','b']]</code>
Select by label	<code>df.loc[rows, cols]</code>
Select by position	<code>df.iloc[rows, cols]</code>
Filter rows	<code>df[df['x']>0]</code> / <code>df.query('x>0')</code>

Sorting & Derived Columns

Task	Code
Sort by column	<code>df.sort_values('x', ascending=False)</code>
Sort by index	<code>df.sort_index()</code>
New column	<code>df['y'] = df['a'] + df['b']</code>
Conditional column (binary)	<code>np.where(cond, val1, val2)</code>
Conditional column (multi)	<code>np.select([c1,c2], [v1,v2], default=v3)</code>
Apply custom function	<code>df['y'] = df['x'].apply(func)</code>
Value lookup / map	<code>df['y'] = df['x'].map(lookup_dict)</code>

GroupBy, Merge & Concat

Task	Code
Group + aggregate	<code>df.groupby('k')['x'].sum()</code>
Multiple aggregates	<code>df.groupby('k').agg(total=('x', 'sum'), avg=('x', 'mean'))</code>
Filter groups (HAVING)	<code>df.groupby('k').filter(lambda g: len(g)>10)</code>
Add group stat to every row	<code>df.groupby('k')['x'].transform('sum')</code>
Join / merge	<code>pd.merge(a, b, on='k', how='left', validate='1:1')</code>
Stack rows (UNION ALL)	<code>pd.concat([a, b])</code>
Stack + dedupe (UNION)	<code>pd.concat([a, b]).drop_duplicates()</code>

Reshaping & Window Functions

Task	Code
Pivot (long -> wide)	<code>pd.pivot_table(df, index='r', columns='c', values='v', aggfunc='sum')</code>
Melt (wide -> long)	<code>df.melt(id_vars=['r'], value_vars=[...])</code>
Cross-tab	<code>pd.crosstab(df['a'], df['b'])</code>
Running total	<code>df['x'].cumsum()</code>
Moving average	<code>df['x'].rolling(7).mean()</code>
Rank within group	<code>df.groupby('k')['x'].rank(method='min')</code>
Prior / next row value	<code>df['x'].shift(1) / .shift(-1)</code>

Cleaning & Type Conversion

Task	Code
Find missing	<code>df.isna().sum()</code>
Drop missing	<code>df.dropna(subset=['x'])</code>
Fill missing	<code>df['x'].fillna(0)</code>
Drop duplicates	<code>df.drop_duplicates(subset=['id'], keep='last')</code>
Cast type	<code>df['x'].astype('float64')</code>
Parse date	<code>pd.to_datetime(df['x'])</code>
Coerce numeric	<code>pd.to_numeric(df['x'], errors='coerce')</code>
Clean strings	<code>df['x'].str.strip().str.lower()</code>

I/O, Visualization & SQL Quick Map

Task	Code / SQL
Read CSV	<code>pd.read_csv('f.csv', parse_dates=['d'])</code>
Read SQL / Snowflake	<code>pd.read_sql(sql, con) / cur.fetch_pandas_all()</code>
Write to Snowflake	<code>write_pandas(conn, df, 'TABLE')</code>
Quick plot	<code>df.plot(kind='bar', x='a', y='b')</code>
Seaborn heatmap	<code>sns.heatmap(df.corr(), annot=True)</code>
SQL: JOIN -> Pandas	<code>.merge(how=...)</code>
SQL: GROUP BY/HAVING -> Pandas	<code>.groupby().agg() + filter</code>
SQL: OVER(PARTITION BY) -> Pandas	<code>.groupby().transform()/.rank()</code>

Closing Notes

- You already have the hardest part — the analytical instinct for tables, keys, and relationships. Pandas is a syntax and execution-model shift, not a new way of thinking.
- Revisit Section 7 (pitfalls) after your first two weeks of real use — that's when index/copy/merge surprises tend to show up.
- Keep this guide open next to your Snowflake worksheet; translate one query a day until it's second nature.